

Testing Guide

Security Enhanced E-Payment System

UIU — Department of CSE | June 2026

This document outlines 6 essential tests for the web-based e-payment system. Each section covers objective, step-by-step procedure, and required tools.

Test 1: Unit Testing — Cryptographic Functions

Objective

Verify that AES-256 encryption/decryption, HMAC generation, and dynamic key derivation ($K_2 + B_p + T$) produce correct and consistent outputs in isolation.

Step-by-Step Process

1. Set up a Python test file using pytest.
2. Write a test for AES-256: encrypt a sample transaction payload, then decrypt it and assert the output matches the original.
3. Write a test for HMAC: generate a hash for a known message with a fixed key, then verify the hash matches the expected value.
4. Write a test for key derivation: pass fixed values for password (K_2), biometric hash (B_p), and timestamp (T) and assert the derived key is deterministic.
5. Write a negative test: alter one byte of the ciphertext and assert decryption fails or produces wrong output.
6. Run pytest and check all assertions pass.

Tools Required

pytest, Python javax.crypto equivalent (PyCryptodome / cryptography library), unittest.mock for biometric input simulation

Test 2: Integration Testing — Transaction Flow (End-to-End)

Objective

Ensure the full transaction pipeline works correctly: user login → transaction creation → AES+HMAC encryption → server decryption → approval/rejection.

Step-by-Step Process

1. Deploy the Django backend locally or on a staging server.
2. Use Postman to simulate a registered user sending a POST request to the /transaction endpoint with a valid encrypted payload.
3. Check the server response: assert HTTP 200 and a transaction approval message.
4. Send another request with a tampered HMAC value and assert the server returns HTTP 400/403 with a rejection message.
5. Send a request with a missing or null biometric field and assert proper error handling.
6. Verify the transaction record is saved correctly in the SQLite/MySQL database after a successful request.

Tools Required

Postman, Django test client (APITestCase), pytest-django, DB Browser for SQLite

Test 3: Security Testing — Replay Attack Prevention

Objective

Confirm that the timestamp + nonce mechanism correctly blocks replayed or duplicate transaction packets.

Step-by-Step Process

1. Capture a valid encrypted transaction payload (using Postman or browser DevTools).
2. Re-send the exact same payload to the /transaction endpoint immediately — server should still accept it (within valid time window).
3. Wait for the timestamp to expire (based on system config, e.g., 30–60 seconds) and re-send the same payload — server must reject it with a 'Stale transaction' error.
4. Without waiting, send the same payload twice in rapid succession — server must reject the second as a duplicate nonce.
5. Attempt to manually modify the timestamp in the payload to a future or past value and confirm the HMAC check fails.
6. Document all server responses and confirm all replay attempts are blocked.

Tools Required

Postman (manual replay), Burp Suite (intercept and replay HTTP requests), Python requests library for scripted replay simulation

Test 4: Performance Testing — Encryption Speed & Load

Objective

Measure that each transaction completes within the 2-second requirement under normal and high-load conditions.

Step-by-Step Process

1. Write a Locust script that simulates a single user completing a full transaction (authenticate + encrypt + send).
2. Run the script with 1 virtual user and record the average response time — it must be under 2 seconds.
3. Gradually increase to 50, 100, and 200 concurrent virtual users.
4. Monitor CPU/memory usage on the server during the load test.
5. Identify the breaking point (where response time exceeds 2s or errors spike).
6. Record results in a table: users vs. avg. response time vs. error rate.

Tools Required

Locust (load testing), Apache JMeter (alternative), Chrome DevTools Network tab (single-user timing), htop / server monitor

Test 5: Offline Transaction Testing — Store and Forward

Objective

Validate that transactions made without internet connectivity are stored securely on the device and synced correctly once connectivity is restored.

Step-by-Step Process

1. Disable network access on the test device or browser (use DevTools → Network → Offline mode).
2. Initiate a transaction through the web app — confirm it is accepted locally and a 'Pending Sync' status is shown.
3. Inspect the local storage (IndexedDB or SQLite) to confirm the encrypted transaction record is saved.
4. Re-enable network access and trigger the sync mechanism (manual button or automatic).
5. Check the backend database to confirm the transaction has been uploaded exactly once (no duplicates).
6. Repeat with multiple offline transactions and verify all sync correctly and in order.

Tools Required

Chrome DevTools (Network offline mode, Application → IndexedDB inspector), DB Browser for SQLite, Django admin panel to verify sync

Test 6: Input Validation & SQL Injection Testing

Objective

Ensure all form inputs are sanitized, and the system is protected against common injection attacks that could compromise the database or authentication flow.

Step-by-Step Process

1. Identify all input fields: registration form, login, and transaction amount fields.
2. Enter a standard SQL injection string (e.g., ' OR '1'=1) in the username/password fields and confirm login fails with a proper error.
3. Enter XSS payloads (e.g., alert(1)) in text fields and confirm output is escaped, not executed.
4. Use OWASP ZAP to run an automated active scan against the web app URL.
5. Review ZAP's report for High/Medium severity findings related to injection, broken authentication, or sensitive data exposure.
6. Fix any identified issues and re-run the scan to confirm resolution.

Tools Required

OWASP ZAP (automated vulnerability scanner), Burp Suite Community (manual input testing), browser DevTools